Dynamic LoR-GS: Adaptive Low-Rank Gaussian Parameters for Memory-Efficient 3D Gaussian Splatting Training

Viriyadhika Putra
University of Toronto
Toronto, Canada
viriyadhika@cs.toronto.edu

Aliza Tarakanov
University of Toronto
Toronto, Canada
alizat@cs.toronto.edu

Cosmo (Bo-Wei) Wen
University of Toronto
Toronto, Canada
cosmo.wen@mail.utoronto.ca

Abstract

3D Gaussian Splatting (3DGS) achieves state-of-the-art novel view synthesis but suffers from prohibitive memory consumption, largely driven by the high-degree Spherical Harmonic (SH) coefficients required for view-dependent color. Current memory-reduction techniques, such as reducing SH degrees or applying uniform low-rank approximations (LoRA), apply uniform constraints to all primitives. This restriction degrades high-frequency specular details while wasting representational capacity on flat, diffuse geometry. In this paper, we present a Dynamic LoRA architecture that heterogeneously allocates parameter capacity across the scene based on physical complexity. By tracking representational demand via accumulated gradients, our system routes Gaussians into discrete rank bins using percentile-based quotas, scaling natively alongside geometric densification. This enables the network to automatically constrain capacity for matte surfaces while prioritizing complex, specular regions. Our evaluation shows that dynamic routing achieves perceptual parity (LPIPS) with full-capacity SH3 baselines at a fraction of the VRAM footprint. Furthermore, it consistently outperforms uniform Static LoRA models operating under the same effective parameter budget, demonstrating the advantage of scene-aware parameter distribution for 3DGS compression.

CCS Concepts

• **Computing methodologies** → **Rendering**; **Image-based rendering**; *Supervised learning by regression*; • **Hardware** → *GPU memory*.

Keywords

3D Gaussian Splatting, Low-Rank Adaptation, novel view synthesis, memory-efficient training, spherical harmonics

1 Introduction

The introduction of 3D Gaussian Splatting (3DGS) [8] has significantly advanced novel view synthesis by providing an explicit, point-based scene representation that achieves real-time rendering at high resolutions. However, this fidelity incurs a substantial computational cost. The explicit nature of 3DGS requires maintaining millions of independent primitives, resulting in a large GPU memory (VRAM) footprint during both training and inference. A majority of this parameter budget is consumed by the higher-order Spherical Harmonic (SH) coefficients, which are necessary to model complex, view-dependent color phenomena such as reflections and specular highlights.

Existing mitigation strategies either operate post-training, which leaves peak VRAM unaffected, or apply uniform constraints that fail to account for the physical heterogeneity of real-world scenes. A matte wall requires minimal view-dependent capacity, whereas a glossy car window requires precise, high-frequency parameterization. Imposing a uniform parameter constraint forces the over-parameterization of diffuse surfaces while limiting the capacity available to render complex specularities accurately.

In this work, we address the limitations of uniform parameter allocation. We propose a Dynamic LoRA architecture that profiles and distributes representational capacity based on the localized demands of the scene. By factorizing the higher-order SH coefficients into a shared projection matrix and per-Gaussian low-rank latents, we create a flexible parameter space. During the geometric densification phase, we track the gradient activity of each primitive to quantify its demand for high-frequency details. Using a percentile-based routing mechanism to avoid spatial starvation, our system dynamically migrates Gaussians between discrete rank bins. This ensures that memory is allocated where the physical topology of the scene demands it.

In summary, our main contributions are:

- We introduce a Dynamic LoRA architecture for 3DGS that factorizes higher-order SH coefficients to enable heterogeneous, per-primitive capacity allocation.
- We develop a routing mechanism based on normalized gradient activity and percentile quotas, allocating higher representational capacity to complex primitives while preventing spatial starvation.
- We demonstrate empirically that our dynamic allocation achieves perceptual parity with full SH3 baselines at significantly reduced VRAM, and consistently outperforms uniform static baselines operating under identical effective parameter budgets.

2 Background

3D Gaussian Splatting (3DGS) [8] is a leading framework for novel view synthesis. By representing scenes with explicit, differentiable 3D Gaussian primitives rendered via fast tile-based rasterization, 3DGS achieves high-fidelity real-time rendering and fast training. This efficiency has motivated extensive follow-on work in dynamic scenes [9], editing [3], compression [1, 11], and avatar generation [12].

Despite this success, optimizing 3DGS on memory-constrained hardware remains challenging due to two coupled factors. First, adaptive densification continuously clones and splits primitives, expanding the population N to several million. Second, each Gaussian requires a substantial parameter budget. Because appearance is view-dependent, a simple RGB representation, using only three floating-point values per point, is insufficient, as it assumes color is constant across all viewing directions. In practice, effects such as

reflections and specular highlights cause color to vary with viewpoint. To capture this, color is modeled using degree-3 spherical harmonic (SH3) coefficients, which provide a compact mathematical representation of direction-dependent color over the viewing sphere. As a result, each primitive stores 48 floating-point values just for color. These coefficients dominate the memory footprint, consuming roughly 576 MB for a 3-million Gaussian scene before accounting for optimizer states and gradients [1, 11]. Consequently, peak training VRAM is a critical bottleneck that existing methods struggle to fully resolve.

3 Related Work

Current memory-reduction strategies generally fall into two categories: pruning and post-training compression. Pruning methods such as Mini-Splatting [4] and LP-3DGS [19] reduce N directly, which can degrade fine geometric detail. Alternatively, compression techniques like Reduced3DGS [11], LFGS [16], SG-Splatting [14], and MEGS^2 [2] reduce the per-Gaussian footprint through SH band masking or lobe replacement. Because these operations occur *after* training completes, the full SH tensor must be maintained throughout optimization, leaving peak training memory unaffected.

Low-Rank Adaptation (LoRA) [6] replaces a parameter matrix $W \in \mathbb{R}^{m\times n}$ with a low-rank factorization $W \approx AB$ (where $A \in \mathbb{R}^{m\times r}$, $B \in \mathbb{R}^{r\times n}$, and $r \ll \min(m, n)$), reducing the number of trainable parameters without materializing the full matrix. The SH tensor $C \in \mathbb{R}^{N\times 48}$ is a natural target: methods like Reduced3DGS [11] successfully prune high-degree bands with minimal quality loss, suggesting the effective rank of C is inherently low for most primitives. Inspired by adaptive budget allocation in language models [17], distributing this rank heterogeneously, rather than applying a uniform constraint, could reduce peak VRAM while preserving perceptual fidelity.

While LoRA has been introduced to the 3DGS ecosystem, it is typically applied to auxiliary neural network weights for dynamic streaming [9] or pose prediction [5]. StyleSplat [7] optimizes SH coefficients directly for style transfer but lacks a low-rank constraint. To our knowledge, this is the first work to apply low-rank factorization directly to 3DGS primitive parameters during optimization to reduce peak training memory.

4 Motivations and Objectives

Driven by the heterogeneity of real-world scenes, our core objective is to decouple a scene's geometric resolution from its view-dependent memory footprint. We aim to design a dynamic, memory-aware routing architecture that autonomously distributes parameter capacity based on localized topological needs in the form of quantized low-rank bins. By continuously evaluating the demand of each primitive and actively readjusting its LoRA rank *during* the densification phase, this project seeks to demonstrate that adaptively allocating memory strictly to the regions that mathematically require it can significantly reduce peak training VRAM while preserving the high-frequency perceptual quality of the rendered scene.

5 Ideas and Methodologies

5.1 Isolating the Memory Bottleneck

Our initial idea was to apply LoRA to any subset of Gaussian attributes (i.e. colors, rotations, scales, or opacities) to directly reduce the parameter budget during training, while leaving all remaining attributes full-rank. Let $G \in \mathbb{R}^{N\times D}$ denote the full attribute matrix, where each row $g^{(i)} \in \mathbb{R}^D$ is the attribute vector of Gaussian i and D is the total per-Gaussian parameter dimension, encompassing attributes such as position, color (SH coefficients), opacity, rotation, and scale. Given a choice of target attributes $\mathcal{T} \subseteq \{\text{position, color, opacity, quat, scale}\}$, let $G_\mathcal{T} \in \mathbb{R}^{N\times D_\mathcal{T}}$ be the corresponding column sub-block of G, where $D_\mathcal{T} \le D$ is the dimension of the targeted subset. Instead of optimizing $G_\mathcal{T}$ directly, we compute a low-rank update:

$$\Delta G_\mathcal{T} = AB, \quad A \in \mathbb{R}^{N\times r},\ B \in \mathbb{R}^{r\times D_\mathcal{T}}, \tag{1}$$

where $r \ll D_\mathcal{T}$ is the rank. A is a per-Gaussian matrix and B is a *shared* global projection matrix, both optimized jointly during training. All columns of G outside $\mathcal{T}$ remain directly optimized and full-rank.

5.1.1 Flexible Target Selection. Because the framework above is attribute-agnostic, we initially explored applying LoRA to various combinations of Gaussian attributes to determine which offer the most effective compression target. For example, with $\mathcal{T} = \{\text{color, quat}\}$, the targeted sub-block is formed by concatenating the corresponding columns of G:

$$G_\mathcal{T} = [G_c \mid G_{\text{quat}}] \in \mathbb{R}^{N\times D_\mathcal{T}}, \tag{2}$$

where $G_c \in \mathbb{R}^{N\times D_c}$ ($D_c = 48$) and $G_{\text{quat}} \in \mathbb{R}^{N\times D_{\text{quat}}}$ ($D_{\text{quat}} = 4$) give $D_\mathcal{T} = 52$. The shared projection $B = [B_c \mid B_{\text{quat}}] \in \mathbb{R}^{r\times D_\mathcal{T}}$ is partitioned accordingly, so the low-rank update decomposes as:

$$\Delta G_\mathcal{T} = AB = [AB_c \mid AB_{\text{quat}}], \tag{3}$$

jointly updating colors and rotations through a single per-Gaussian latent matrix A. All remaining attributes of G (positions, opacity, scale) fall outside $\mathcal{T}$ and are optimized directly. Because the frozen base columns of $G_\mathcal{T}$ lack gradients, no Adam optimizer states are allocated for them.

5.1.2 LoRA-Aware Densification and Computation. Standard densification must be wrapped with LoRA-aware bookkeeping: cloning or splitting a Gaussian duplicates its corresponding row in A, while pruning removes it. During training, $G_{\mathcal{T},\text{base}} + \Delta G_\mathcal{T}$ is computed dynamically and passed to the rasterizer, ensuring regularization losses correctly penalize the effective parameters influencing the render.

5.1.3 Settling on Higher-Order SH. Ablation studies revealed that applying LoRA to geometry (positions, scales, rotations) degrades reconstruction quality, as these require precise, high-rank updates to represent sharp structural boundaries. Therefore, we restrict LoRA to higher-order SH bands (1–3), approximating only the color matrix $G_c \in \mathbb{R}^{N\times D_c}$ with a single fixed rank r uniformly across all Gaussians: $G_c = AB$, $A \in \mathbb{R}^{N\times r}$, $B \in \mathbb{R}^{r\times D_c}$. Because this tensor dominates the per-Gaussian parameter budget, it is simultaneously the most effective target for memory reduction (see Table 2). We refer to this uniform-rank baseline as **Static LoRA**.

5.2 Dynamic Resource Allocation

We partition the N Gaussians into $B = 3$ bins with rank assignments $r_b \in \{2, 8, 32\}$, where bin b contains N_b Gaussians with $\sum_{b=1}^{B} N_b = N$. The LoRA target is the higher-order SH color coefficients (bands 1–3); the zero-order SH and all geometry parameters remain full-rank throughout. Let $G_{c,b} \in \mathbb{R}^{N_b \times D_c}$ be the color submatrix restricted to bin b, where D_c is the dimension of the bands 1–3 coefficients. It is reconstructed directly via a bin-specific low-rank factorization:

$$G_{c,b} = A_b B_b, \quad A_b \in \mathbb{R}^{N_b \times r_b},\ B_b \in \mathbb{R}^{r_b \times D_c}, \tag{4}$$

where r_b controls the representational capacity of bin b, keeping the per-bin memory footprint strictly equal to $N_b r_b + r_b D_c$. Bin fractions (N_b/N) are scene-specific, estimated from a single color variance pass at step 4000 using the current Gaussian positions and training camera poses. Assigning different ranks r_b to different bins allows different subsets of Gaussians to use different levels of expressiveness, decoupling per-primitive capacity from the global parameter budget.

5.2.1 Global Masking as a Proof of Concept. To validate the rasterizer's ability to handle heterogeneous capacities, we first implemented a global masking strategy. We allocated a single high-rank tensor for the entire scene and simulated dynamic allocation by applying a binary mask during the forward pass to silence higher-order dimensions for selected Gaussians. While the approach is VRAM-inefficient due to the underlying dense allocation, this confirmed that heterogeneous rank representations maintain training stability, justifying the development of a genuinely sparse architecture.

5.2.2 Scoring Signals. We evaluated three per-Gaussian scoring signals to quantify representational demand. The **gradient score** accumulates the absolute gradients of the loss with respect to the active LoRA parameters:

$$\gamma^{(i)} \mathrel{+}= \left\| \nabla_{a^{(i)}} \mathcal{L} \right\|_1 \tag{5}$$

A persistently high gradient indicates the current rank is insufficient to capture the Gaussian's view-dependent appearance. Post-epoch, this value is normalized by the current rank ($\tilde{\gamma}^{(i)} = \gamma^{(i)}/r^{(i)}$) to enable cross-bin comparison, and the buffer is reset. We also evaluated an **opacity-weighted gradient** ($s_i^{\text{grad}}/(\alpha_i + \varepsilon)$), which corrects for near-transparent Gaussians that contribute little to the rendered image regardless of SH complexity, and **SH energy** ($\|a^{(i)}B\|_F^2$), which measures the total magnitude of the reconstructed higher-order SH color coefficients for Gaussian i. We additionally explored fully learnable rank assignment via SGD on per-Gaussian rank values and via soft per-Gaussian logits with a rank regularization loss; both collapsed toward a single bin and were not pursued further.

Across all scoring signal combinations, rendering quality differences were negligible (PSNR range 24.33–24.44 dB), confirming that quota distribution matters more than the choice of scoring signal.

5.2.3 Absolute Thresholds. Our initial active migration strategy promoted or demoted ranks using absolute scalar thresholds on $\gamma^{(i)}$ — for example, fixed cutoffs ($\gamma^{(i)} > 10^{-3}$) or median-relative bounds ($\gamma^{(i)} > k \cdot \text{median}(\gamma)$). Empirical testing demonstrated this approach is brittle: gradient magnitudes vary significantly across datasets, so loose bounds produced static allocations while tight bounds induced rank thrashing. This establishes that absolute thresholding is ill-suited for the dynamic topology of 3DGS.

5.2.4 Percentile Routing. To map $\tilde{\gamma}^{(i)}$ scores to bin assignments, we first tried fixed numerical quotas (e.g. 100K Gaussians to $r = 2$, 200K to $r = 8$, remainder to $r = 32$), which allow static tensor pre-allocation. However, both fixed and percentile routing share a "materialization cost" floor, negating this advantage. We therefore use percentile quotas ($q_{\text{top}}, q_{\text{mid}}, q_{\text{bot}}$) (e.g. (0.4, 0.4, 0.2) for the high-, mid-, and low-rank bins respectively), which scale natively with the evolving geometry as densification grows N. We refer to this method as **Fixed Percentile LoRA**.

5.2.5 Scene Analysis for Relative Thresholding. Rather than treating ($q_{\text{top}}, q_{\text{mid}}, q_{\text{bot}}$) as arbitrary hyperparameters, we developed a method to estimate scene-specific quotas automatically early in training, prior to reaching peak GPU memory. This is necessary because GPU memory grows continuously during densification, peaking at approximately step 6100 in standard SH3 training, so any quota decision must be made before this point. Inspired by post-training variance analysis [11], we evaluated two predictive signals during the densification phase.

SH energy ($\|a^{(i)}B\|_F^2$) measures the total magnitude of the reconstructed higher-order SH color coefficients, but depends on the learned parameters and is therefore sensitive to warmup rank: under rank-2 warmup, it remains unreliable until step 6000 — past the memory peak.

Color variance measures photometric consistency of each Gaussian across training views:

$$s_i^{\text{color}} = \frac{1}{3}\left(\text{Var}(R_i) + \text{Var}(G_i) + \text{Var}(B_i)\right) \tag{6}$$

Each Gaussian is projected into all training images using its current 3D position and SfM camera parameters; pixel colors are sampled at the projected locations. Because this signal depends only on Gaussian geometry and not on learned parameters, it is unaffected by warmup rank and converges to low prediction error (∼0.035) by step 1100, well before the memory peak (Figure 1).

We evaluate the predictive accuracy of these signals against a step-15,000 full-SH3 gold standard, using GMM clustering to determine boundaries automatically. As shown in Figure 1, color variance converges to low prediction error (∼0.035) by step 1100 and remains stable regardless of warmup rank, while SH energy under rank-2 warmup remains unreliable until step 6000 — past the memory peak.

Figure 1 compares prediction error over training steps for color variance and SH energy, evaluated under rank-2 and rank-32 warmups. Because color variance depends solely on Gaussian geometry, it is unaffected by warmup rank and consistently outperforms SH energy. Conversely, SH energy begins with large errors because the LoRA factors lack meaningful initialization. While the rank-32 model recovers by step 2100, the rank-2 model remains volatile until step 6000, and both remain less accurate than color variance.

We therefore train at rank 2 until step 4000 and run a single color variance pass to establish scene-specific quotas, minimizing memory consumption through the densification phase while providing

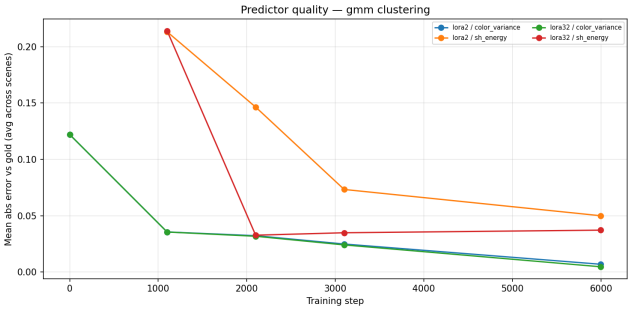

Figure 1: Mean absolute error vs. gold standard (GMM, averaged across scenes). Color variance (blue/green) remains below SH energy (orange/red) at all steps, including step 0.

a reliable estimate before peak memory is reached. We refer to this method as **Dynamic LoRA**.

6 Experimental Setup

All experiments were conducted on a single NVIDIA GeForce RTX 4070 Ti (12 GB VRAM) running Ubuntu 24.04.4 LTS with Python 3.12.13, PyTorch 2.11.0, CUDA 12.8, and gsplat 1.5.3. We evaluate on three scenes from the Ref-NeRF [13] real dataset (*gardenspheres*, *sedan*, and *toycar*), downsampled by a factor of 4. All variants are trained for 15,000 steps with Gaussian densification stopped at step 10,000, a batch size of 1, and an SSIM loss weight of 0.2. Each variant is run three times per scene (45 total runs). Models are evaluated at steps 7,000 and 15,000 using PSNR, SSIM [15], and LPIPS [18] (AlexNet).

Five model variants are compared:

- **SH3** — full degree-3 SH, no compression. Gold standard upper bound.
- **SH2** — full degree-2 SH, no compression. Low-memory baseline (15 parameters per channel).
- **Static LoRA** — uniform LoRA rank $r = 16$ across all Gaussians.
- **Fixed Percentile LoRA** — dynamic routing into bins $r \in \{2, 8, 32\}$ with fixed scene-independent quotas: bottom 20% $\rightarrow r = 2$, middle 40% $\rightarrow r = 8$, top 40% $\rightarrow r = 32$. Weighted average rank: $0.2\times2 + 0.4\times8 + 0.4\times32 = 16.4$.
- **Dynamic LoRA** — same bins and routing as Fixed Percentile LoRA, with scene-specific quotas estimated via GMM clustering on color variance at step 4000.

To ensure a fair budget comparison, Static LoRA ($r = 16$), Fixed Percentile LoRA, and Dynamic LoRA all operate at a weighted average rank of $r_{\text{eff}} \approx 16$–$16.4$.

7 Results

To evaluate the computational and perceptual trade-offs of our architecture, we assess models across four primary metrics: Peak training VRAM (GB), PSNR, SSIM, and LPIPS (AlexNet). Because PSNR inherently favors over-smoothed pixel averages, we prioritize LPIPS as the primary indicator of preserved high-frequency, view-dependent specularities. Within our evaluation context, meaningful algorithmic improvements are defined as VRAM reductions of ≥ 0.5 GB, PSNR deltas of ≥ 0.1 dB, and LPIPS reductions of ≥ 0.01.

7.1 Overall Performance and Memory Reduction

Table 1 presents the quantitative evaluation of our Dynamic LoRA architecture against the full-capacity Gold Standard (SH3) and a uniform Static LoRA baseline. Across all datasets, our dynamic routing achieves an optimal quality-to-memory tradeoff, significantly reducing the peak training VRAM footprint while preserving perceptual fidelity.

Compared to the SH3 baseline, our method achieves substantial memory savings: reducing peak VRAM by $\sim 26\%$ on the heavily populated *Garden Sphere* scene (7.08 $\rightarrow$ 5.18 GB) and up to 36% on the *Sedan* scene (2.90 $\rightarrow$ 1.85 GB). Notably, despite the bookkeeping overhead of maintaining dynamic routing indices, our method achieves a memory footprint that is either highly competitive with or strictly superior to the uniform Static LoRA baseline (e.g., achieving the lowest footprint on the *Sedan* dataset).

Importantly, this VRAM reduction does not incur a proportional penalty to rendering quality. As shown in Table 1, Dynamic LoRA consistently secures the "best" or "second-best" position across most quantitative metrics. In terms of PSNR and SSIM, our architecture closely matches the unconstrained SH3 baseline, demonstrating that dynamically constraining parameters does not destabilize geometric convergence. Furthermore, on the highly complex *Garden Sphere* dataset, our dynamic method outperforms the SH3 Gold Standard in LPIPS (0.217 vs. 0.219). This validates our core architectural hypothesis: by adaptively allocating parameter capacity strictly to regions that demand it, the network is forced to learn robust structural representations rather than overfitting high-degree Spherical Harmonics to noise, thereby maintaining sharp, view-dependent realism at a fraction of the hardware cost.

7.2 Attribute Ablation for LoRA Targeting

We evaluate the effect of LoRA target selection on reconstruction quality (PSNR) across three scenes (*Garden Sphere*, *Sedan*, *Toycar*). The full-rank SH3 baseline operates without any LoRA constraint.

As shown in Table 2, all LoRA configurations remain competitive with the full-rank baseline, maintaining quality gaps generally under 0.5 dB. Isolating LoRA strictly to colors achieves the closest match to the baseline on the *Sedan* scene (25.14 vs. 25.16 dB) and performs consistently across all scenes. Expanding the target to include rotations yields nearly identical results, suggesting rotations possess an inherent low-rank structure that LoRA captures without significant loss.

Conversely, including scales in the target set introduces the most notable degradation. Configurations targeting scales drop by approximately 0.75 dB on *Sedan* compared to the colors-only approach, indicating that scale parameters require full-rank updates to preserve fine geometric detail. Furthermore, targeting solely geometry attributes (rotations and scales) achieves competitive PSNR on *Garden Sphere* and *Toycar* but underperforms on *Sedan*, highlighting a scene-dependent sensitivity to geometric factorization.

Dataset	Method	Peak VRAM (GB) ↓	PSNR (db)↑	SSIM ↑	LPIPS ↓
Garden Sphere	SH3 (Gold Standard)	7.08 ± 0.00	**22.899 ± 0.046**	**0.614 ± 0.000**	0.219 ± 0.001
	Static LoRA	**4.74 ± 0.01**	22.865 ± 0.001	0.614 ± 0.000	0.225 ± 0.000
	Dynamic LoRA (Ours)	5.18 ± 0.01	22.875 ± 0.009	0.612 ± 0.000	**0.217 ± 0.000**
Sedan	SH3 (Gold Standard)	2.90 ± 0.01	25.430 ± 0.019	0.719 ± 0.000	0.274 ± 0.000
	Static LoRA	1.96 ± 0.00	**25.443 ± 0.030**	**0.720 ± 0.000**	**0.274 ± 0.000**
	Dynamic LoRA (Ours)	**1.85 ± 0.02**	25.427 ± 0.016	0.718 ± 0.000	0.276 ± 0.000
Toycar	SH3 (Gold Standard)	3.42 ± 0.00	**24.819 ± 0.005**	**0.678 ± 0.000**	**0.192 ± 0.001**
	Static LoRA	**2.40 ± 0.01**	24.663 ± 0.002	0.673 ± 0.000	0.196 ± 0.000
	Dynamic LoRA (Ours)	2.41 ± 0.01	24.771 ± 0.007	0.677 ± 0.000	0.194 ± 0.001

Table 1: Overall Performance and Memory Reduction. Bold indicates the best result, and underline indicates the second-best per dataset.

Figure 2: Qualitative Evaluation and Dynamic Rank Distribution on the *Sedan* Scene. (Left to right: Ground Truth, SH3 Baseline, Dynamic LoRA (Ours), Dynamic Rank Heatmap). Our Dynamic LoRA architecture achieves perceptual parity with the unconstrained SH3 baseline, preserving high-frequency details and specular reflections. The rank heatmap visualizes our dynamic allocation strategy: the system constrains diffuse backgrounds to a minimal capacity (r_{min}, blue), allocates moderate capacity to structural edges (r_{mid}, purple), and concentrates its maximum parameter budget on complex, view-dependent textures (r_{max}, red). This demonstrates that our routing mechanism actively learns and exploits the physical topology of the scene.

Overall, targeting only colors provides the most predictable and robust behavior across varying topologies, justifying its selection as the exclusive target for our dynamic rank architecture.

Target	Garden Sphere	Sedan	Toycar
Full-rank SH3 (Baseline)	23.29	25.16	25.01
Colors Only (SH 1–3)	23.08	**25.14**	24.85
Colors + Rotations	**23.10**	25.02	**24.87**
Colors + Opacities	22.95	25.05	24.78
Colors + Scales	22.94	24.40	24.81
Colors + Rot. + Scales	22.94	24.44	24.81
Rot. + Scales (Ref.)	23.14	24.30	24.98
Rot. + Scales + Opac. (Ref.)	23.12	24.35	25.05

Table 2: Ablation of LoRA target configurations on reconstruction quality (PSNR (db)). Bold indicates the best result among color-targeting configurations per scene. Geometry-only rows serve as baselines and are not candidates for dynamic memory reduction.

7.3 Effective Rank Analysis and Architectural Trade-offs

To isolate the specific impact of our dynamic routing mechanism, we evaluate our architecture against Static LoRA and SH2 under a matched average parameter budget (see Experimental Setup).

By architectural definition, standard SH2 establishes the lower bound for memory consumption (e.g., 4.50 GB on the *Garden Sphere* scene). Because LoRA-based methods require maintaining localized pointer arrays, bin indices, and segregated Adam optimizer states, they inherently carry a VRAM overhead that prevents them from outperforming SH2 in pure memory reduction at the same effective rank.

However, evaluating these representations strictly through global quantitative metrics (PSNR and SSIM) obscures the critical advantage of heterogeneous allocation. While all methods achieve effective parity in these global averages, spatial averaging dilutes localized perceptual improvements. Standard SH2 and Static LoRA constrain every primitive to a uniform capacity, limiting the reconstruction of complex, high-frequency specularities. Consequently, while they save memory, they tend to over-smooth fine details.

In contrast, our dynamic architecture leverages the same average parameter budget but distributes it heterogeneously. As demonstrated by the LPIPS metric, which heavily penalizes the loss of

Dataset	Method	Peak VRAM (GB) ↓	PSNR (db)↑	SSIM ↑	LPIPS ↓
Garden Sphere	SH2 (Low-Memory Baseline)	**4.50 ± 0.00**	**22.938 ± 0.009**	**0.616 ± 0.000**	0.225 ± 0.001
	Fixed Percentile LoRA	5.77 ± 0.01	22.881 ± 0.005	0.612 ± 0.000	**0.217 ± 0.001**
	Static (Uniform Rank)	4.74 ± 0.01	22.865 ± 0.001	0.614 ± 0.000	0.225 ± 0.000
Sedan	SH2 (Low-Memory Baseline)	**1.85 ± 0.02**	25.328 ± 0.018	0.716 ± 0.000	0.277 ± 0.000
	Fixed Percentile LoRA	2.35 ± 0.01	25.431 ± 0.013	0.719 ± 0.000	0.274 ± 0.001
	Static (Uniform Rank)	1.96 ± 0.00	**25.443 ± 0.030**	**0.720 ± 0.000**	**0.274 ± 0.000**
Toycar	SH2 (Low-Memory Baseline)	**2.23 ± 0.00**	**24.805 ± 0.008**	**0.680 ± 0.000**	0.196 ± 0.000
	Fixed Percentile LoRA	2.86 ± 0.01	24.750 ± 0.006	0.675 ± 0.000	**0.193 ± 0.000**
	Static (Uniform Rank)	2.40 ± 0.01	24.663 ± 0.002	0.673 ± 0.000	0.196 ± 0.000

Table 3: Ablation of effective rank and architectural trade-offs. Bold indicates the best result, and underline indicates the second-best per dataset.

sharp, view-dependent textures, dynamic routing consistently outperforms both SH2 and Static LoRA on complex scenes. By constraining flat, diffuse geometry that requires minimal capacity, the system frees parameters to prioritize complex specular regions. This demonstrates that dynamic rank distribution is structurally superior to a static uniform rank: it matches the physical topology of the scene, achieving higher perceptual realism without requiring a larger global parameter budget.

8 Discussion and Takeaways

8.1 Active Allocation vs. Post-Hoc Pruning

Mallick et al. [10] address peak training VRAM by replacing unconstrained densification with a budget-constrained constructive process. While this systematically degrades quality as Gaussian counts (N) shrink, it primarily targets geometric redundancy. Our work is fundamentally orthogonal, targeting representation redundancy within the SH coefficients. Because high spatial frequency does not strictly correlate with high view-dependency, these approaches are complementary. As Papantonakis et al. [11] establish that primitive count and SH storage are the two dominant memory bottlenecks in 3DGS, a hybrid approach is highly promising. Combining budget-constrained densification with our dynamic LoRA rank assignment would enable a dual-budget optimization framework. Such a system could allocate resources precisely by spending the "Gaussian budget" on geometric detail and the "Rank budget" on complex view-dependent effects. This approach would likely achieve a more favorable Pareto frontier than either method in isolation.

8.2 Material-Driven Limits and the Oracle Problem

A critical limitation arises in globally uniform environments (e.g., the matte *Toycar* dataset), where perfectly aligned native representations (e.g., universal SH1) outperform our dynamic architecture by avoiding tensor slicing and routing overhead. This exposes an oracle problem in memory-efficient 3DGS: mathematically optimizing the percentile quotas ($q_{\text{top}}, q_{\text{bot}}$) requires prior knowledge of the scene's final material complexity. However, because geometry is volatile during early densification, this complexity is only revealed *by* training. This creates a functional impasse: perfectly configuring the memory-saving architecture requires a full training run, which inherently renders the architecture itself redundant. Resolving this paradox necessitates future work into pre-optimization estimators capable of predicting scene complexity directly from the initial sparse point cloud.

8.3 Implementation Overhead and Future Integration

Despite requiring per-bin optimizers and periodic reallocation, our implementation achieves memory savings close to the theoretical limit. For the *Garden Sphere* scene, our average rank of 16.4 predicts a theoretical saving of 1.48 GB against a measured 1.31 GB. This 0.17 GB gap (similarly minimal in *Sedan* and *Toycar*) is achieved through strict Python-level memory management, explicitly clearing stale optimizer states during rank reallocation. The remaining discrepancy represents a framework-level materialization cost. Specifically, temporary dense tensors must be allocated during densification and forward passes to interface fragmented bin matrices with standard framework logic. To completely eliminate this indexing overhead and maximize hardware efficiency, variable-rank memory pointers must eventually be integrated into custom C++/CUDA kernels. Nevertheless, our implementation validates the architectural viability of dynamic capacity routing.

9 Conclusion

We demonstrated that the prohibitive memory footprint of 3DGS is primarily a consequence of uniform parameter allocation. Our Dynamic LoRA architecture employs gradient-based percentile routing to identify and exploit heterogeneous representational demands, automatically constraining diffuse geometry to prioritize capacity for complex specularities. Consequently, our method achieves the perceptual realism of a full-capacity SH3 model at a fraction of the memory cost. While current framework-level tensor management imposes a minor materialization overhead, the architectural viability of dynamic routing motivates the next critical step: integrating variable-rank parameter resolution directly into custom CUDA kernels. By aligning mathematical capacity with physical scene topology, we provide a scalable foundation for memory-efficient 3DGS architectures.

References

[1] Milena T. Bagdasarian, Paul Knoll, Yi-Hsin Li, Florian Barthel, Anna Hilsmann, Peter Eisert, and Wieland Morgenstern. 2025. 3DGS.zip: A survey on 3D Gaussian Splatting Compression Methods. *Comput. Graph. Forum* 44, 2 (2025). doi:10.1111/CGF.70078

[2] Jiarui Chen, Yikeng Chen, Yingshuang Zou, Ye Huang, Peng Wang, Yuan Liu, Yujing Sun, and Wenping Wang. 2025. MEGS^2: Memory-Efficient Gaussian Splatting via Spherical Gaussians and Unified Pruning. *CoRR* abs/2509.07021 (2025). doi:10.48550/ARXIV.2509.07021 arXiv:2509.07021

[3] Yiwen Chen, Zilong Chen, Chi Zhang, Feng Wang, Xiaofeng Yang, Yikai Wang, Zhongang Cai, Lei Yang, Huaping Liu, and Guosheng Lin. 2024. GaussianEditor: Swift and Controllable 3D Editing with Gaussian Splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*. IEEE, 21476–21485. doi:10.1109/CVPR52733.2024.02029

[4] Guangchi Fang and Bing Wang. 2024. Mini-Splatting: Representing Scenes with a Constrained Number of Gaussians. In *Computer Vision - ECCV 2024 - 18th European Conference, Milan, Italy, September 29-October 4, 2024, Proceedings, Part LXXVII (Lecture Notes in Computer Science)*, Ales Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol (Eds.). Springer, 165–181. doi:10.1007/978-3-031-72980-5_10

[5] Yuki Fujimura, Takahiro Kushida, Kazuya Kitano, Takuya Funatomi, and Yasuhiro Mukaigawa. 2025. UFV-Splatter: Pose-Free Feed-Forward 3D Gaussian Splatting Adapted to Unfavorable Views. *CoRR* abs/2507.22342 (2025). doi:10.48550/ARXIV.2507.22342 arXiv:2507.22342

[6] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net. https://openreview.net/forum?id=nZeVKeeFYf9

[7] Sahil Jain, Avik Kuthiala, Prabhdeep Singh Sethi, and Prakanshul Saxena. 2024. StyleSplat: 3D Object Style Transfer with Gaussian Splatting. *CoRR* abs/2407.09473 (2024). doi:10.48550/ARXIV.2407.09473 arXiv:2407.09473

[8] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Trans. Graph.* 42, 4 (2023), 139:1–139:14. doi:10.1145/3592433

[9] Zhenhuan Liu, Shuai Liu, Yidong Lu, Yirui Chen, Jie Yang, and Wei Liu. 2025. Efficient 4D Gaussian Stream with Low Rank Adaptation. *CoRR* abs/2502.16575 (2025). doi:10.48550/ARXIV.2502.16575 arXiv:2502.16575

[10] Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De la Torre. 2024. Taming 3DGS: High-Quality Radiance Fields with Limited Resources. In *SIGGRAPH Asia 2024 Conference Papers, SA 2024, Tokyo, Japan, December 3-6, 2024*, Takeo Igarashi, Ariel Shamir, and Hao (Richard) Zhang (Eds.). ACM, 2:1–2:11. doi:10.1145/3680528.3687694

[11] Panagiotis Papantonakis, Georgios Kopanas, Bernhard Kerbl, Alexandre Lanvin, and George Drettakis. 2024. Reducing the Memory Footprint of 3D Gaussian Splatting. *Proc. ACM Comput. Graph. Interact. Tech.* 7, 1 (2024), 16:1–16:17. doi:10.1145/3651282

[12] Shenhan Qian, Tobias Kirschstein, Liam Schoneveld, Davide Davoli, Simon Giebenhain, and Matthias Nießner. 2024. GaussianAvatars: Photorealistic Head Avatars with Rigged 3D Gaussians. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*. IEEE, 20299–20309. doi:10.1109/CVPR52733.2024.01919

[13] Dor Verbin, Peter Hedman, Ben Mildenhall, Todd E. Zickler, Jonathan T. Barron, and Pratul P. Srinivasan. 2022. Ref-NeRF: Structured View-Dependent Appearance for Neural Radiance Fields. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*. IEEE, 5481–5490. doi:10.1109/CVPR52688.2022.00541

[14] Yiwen Wang, Siyuan Chen, and Ran Yi. 2025. SG-Splatting: Accelerating 3D Gaussian Splatting with Spherical Gaussians. *CoRR* abs/2501.00342 (2025). doi:10.48550/ARXIV.2501.00342 arXiv:2501.00342

[15] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE Trans. Image Process.* 13, 4 (2004), 600–612. doi:10.1109/TIP.2003.819861

[16] Ruiqi Wu, Linlin Jiao, Gang Liu, Li Zhu, Xuan Fei, Yashuang Mu, and Chao Fan. 2025. LFGS: A lightweight framework for efficient 3D Gaussian Splatting with minimal memory footprint. *Comput. Graph.* 131 (2025), 104309. doi:10.1016/J.CAG.2025.104309

[17] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. 2023. Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=lq62uWRJjiY

[18] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. 2018. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. In *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*. Computer Vision Foundation / IEEE Computer Society, 586–595. doi:10.1109/CVPR.2018.00068

[19] Zhaoliang Zhang, Tianchen Song, Yongjae Lee, Li Yang, Cheng Peng, Rama Chellappa, and Deliang Fan. 2024. LP-3DGS: Learning to Prune 3D Gaussian Splatting. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/dd51dbce305433cd60910dc5b0147be4-Abstract-Conference.html